

A Controlled Reassessment of Support Vector Regression on the California Housing Benchmark with Feature Scaling, Feature Selection and Hyperparameter Tuning

Emmanuel Adutwum^{1*}

^{1*}Soka University of America, Aliso Viejo, California, United States.

Corresponding author(s). E-mail(s): eadutwum@soka.edu;

Support Vector Regression (SVR) was recently reported as the weakest of six regressors on the California Housing benchmark by Preethi et al. (2025), who obtained an R^2 of approximately 0.15 for SVR with an RBF kernel while linear, ridge and polynomial-ridge models reached approximately 0.58. This paper asks whether that outcome is a property of the algorithm or of the configuration in which it was run, and answers the question by reproduction rather than by assertion. The configuration described in the prior study — eight raw features, an RBF kernel, a grid over C and ε only, and no scaling step — is re-run here: it yields $R^2 = -0.020$ with library defaults and $R^2 = 0.514$ when C and ε are tuned, bracketing the published value, whereas adding standardisation to the identical pipeline yields $R^2 = 0.766$. A four-factor ablation over scaling, derived features, feature selection and hyperparameter tuning is then evaluated in every one of its 32 cells, and each component is attributed an order-independent Shapley value; the apparent contribution of scaling varies by 0.7441 in R^2 across orderings, which is why single-path ablations should not be read as independent contributions. All ten comparison models are retrained with identical features, identical training rows and an identical tuning budget; differences are tested on 10 common splits using the Nadeau–Bengio corrected resampled t -test, and the confidence interval formula used in the earlier version of this work is corrected. Spatially blocked cross-validation and feature selection performed inside every fold quantify how optimistic the random-split estimate is. The workflow is then applied unchanged to King County and Ames housing data, where the ordering scaling > tuning > selection is reproduced under uniform standardisation on all 3 datasets but on only 1 under the distribution-aware scaler rule proposed here. Three negative results

are reported: the hand-assigned feature-specific scaling strategy does not outperform plain standardisation, its rule-based successor does not transfer to the other datasets and is harmful on the smallest of them, and the tuned SVR remains behind gradient-boosted trees even at equal training size. Conclusions are restricted to the datasets and configurations examined here.

1 Introduction

1.1 Background

Estimating residential property value sits at the boundary of economics, urban planning and statistical learning. For several decades the dominant approach was the hedonic pricing model, which decomposes the value of a property into the implicit prices of its attributes [1]. Hedonic regression offers an interpretable and theoretically grounded structure, but its linearity assumption becomes restrictive as datasets grow and the relationships between attributes and prices become less additive. Machine learning offers an alternative in which flexible function approximators are fitted without explicit distributional assumptions [2–4].

The California Housing dataset, derived from the 1990 United States Census and introduced to the statistical literature by Pace and Barry [5], has become one of the most widely used regression benchmarks. It contains 20,640 census block groups described by eight numerical features, with the median house value of the block group as the target. It is distributed with scikit-learn [6], which makes any experiment on it straightforward to reproduce — and therefore straightforward to check.

1.2 The result under examination

Preethi et al. [7] compared six regression models on this dataset: linear regression, ridge regression, best polynomial ridge regression, lasso regression, elastic net regression, and SVR with an RBF kernel. They report that SVR performs worst, with the highest mean squared error and the lowest R^2 , and that the linear, ridge and polynomial-ridge models each explain approximately 60% of the variance.

Two features of that report matter here. First, the figure of approximately 0.60 is the score of the *linear family*, not of SVR: reading their Figs. 7 and 8, the SVR-RBF model attains $R^2 \approx 0.15$ with a mean squared error of approximately 1.12, against $R^2 \approx 0.58$ and $\text{MSE} \approx 0.55$ for the leading models. An earlier version of the present work quoted 0.60 as the SVR result; that reading was incorrect and is corrected here. Second, their methodology section describes loading the data, splitting it, fitting the six models, tuning SVR’s C and ϵ with a grid search, and evaluating with k -fold cross-validation. No feature scaling step is described anywhere in the paper. Since the RBF kernel acts on Euclidean distances between feature vectors, a pipeline in which `Population` ranges over [3, 35,682] while `AveBedrms` ranges over [0.33, 34] places almost all of the kernel’s sensitivity on a single feature.

1.3 Research questions

This study is a controlled reassessment of that specific result, not a general claim about SVR. It asks four questions:

- Q1 Can the reported SVR result be reproduced from the configuration described, and what does the identical pipeline yield when a scaling step is added? (Section 5.1)
- Q2 When every model is given the same features, the same training rows and the same tuning budget, where does a tuned SVR actually rank, and are the differences statistically distinguishable? (Sections 5.2 and 5.3)
- Q3 How much of the improvement is attributable to each pipeline component once the attribution is made independent of the order in which components are applied? (Section 5.4)
- Q4 How much of the measured performance survives spatially blocked validation and feature selection performed inside the resampling loop, and does the pattern hold on other datasets? (Sections 5.5 and 5.6)

1.4 Contributions

The contribution of this paper is methodological and diagnostic rather than algorithmic. No new learning algorithm is proposed. Specifically:

1. A corrected reading of the prior result, and a direct reproduction of the configuration described there, run over several seeds and reported against the published values (Section 5.1).
2. An explicit statement of the preprocessing workflow as a staged procedure with defined inputs and outputs (Section 3), including a distribution-aware rule that assigns a scaler to each feature from training-fold statistics alone (Algorithm 1). In the earlier version this assignment was a hand-made table specific to one dataset; stating it as a rule makes it auditable and, more importantly, testable on other data — which is how Section 5.6 establishes that it does not transfer.
3. An order-independent ablation procedure for preprocessing pipelines: the full factorial design is evaluated and each component receives a Shapley value, which removes the dependence of the reported increments on the sequence in which components are introduced (Section 3.7).
4. An evaluation protocol with a corrected confidence interval, paired tests on identical splits using the Nadeau–Bengio correction [8], spatially blocked cross-validation [9], and feature selection repeated inside every fold [10, 11].
5. Three negative results, reported because they qualify the earlier version’s claims and this one’s: the feature-specific scaling strategy does not outperform uniform standardisation on this dataset; the distribution-aware scaler rule that replaced it does not transfer, holding the component ordering on only 1 of 3 datasets against 3 for plain standardisation, and carrying a negative Shapley value on Ames; and the derived features contribute little once selection and tuning are in place.

1.5 Scope of the claims

All quantitative claims in this paper are restricted to the datasets, feature sets, search spaces and implementations described in Section 4. That feature scaling is necessary for distance-based kernels is a long-established result [12–14]; this paper does not claim it as a discovery. What is offered is a controlled measurement of how much of one specific published comparison is attributable to that omission, and a protocol under which such a measurement can be repeated.

2 Related work

2.1 Feature scaling in kernel methods

The sensitivity of kernel methods to input scaling is documented from the earliest practical guides onward. Hsu et al. [13] make normalisation the first step of their recommended SVM procedure; Smola and Schölkopf [15] note that the kernel is a function of distances in input space and therefore inherits any anisotropy in the feature scales. Sola and Sevilla [12] demonstrated the effect for neural networks on industrial data, and Singh and Singh [14] survey normalisation techniques and quantify their effect across classifiers. Fernández-Delgado et al. [16] report, over 121 datasets, that kernel machines are considerably more sensitive to preprocessing quality than tree ensembles, which are invariant to monotone transformations of individual features. The present study is positioned within this literature as a reassessment of one published comparison, not as a new finding about scaling.

2.2 The California Housing benchmark

Pace and Barry [5] introduced the data in a spatial-autoregression study and reported that spatial dependence carries substantial predictive information. Subsequent comparative studies routinely place gradient-boosted trees at the top of the ranking on this benchmark [17]. Preethi et al. [7] are the immediate point of comparison for this work; their study is discussed in Section 1.2 and reproduced in Section 5.1. Because the dataset is a geographic sample, the validation design matters: neighbouring block groups are strongly dependent, so a random partition places near-duplicates on both sides of the split. Roberts et al. [9] and Meyer et al. [18] show that random cross-validation is optimistic for spatially structured data and recommend blocked designs, which are adopted in Section 5.5.

2.3 Comparing models and attributing gains

Comparing learners on a single test split conflates a genuine difference with partition noise. Dietterich [19] analyses the error rates of common comparison procedures, and Nadeau and Bengio [8] show that the ordinary paired t -test applied to repeated random splits is anti-conservative because the resampled training sets overlap; they derive the variance correction used in Section 5.3. Demšar [20] extends the discussion to comparisons over many datasets. On the selection side, Ambroise and McLachlan

[11] and Varma and Simon [10] show that performing feature selection or model selection outside the resampling loop produces optimistically biased estimates, and Cawley and Talbot [21] document the same effect for hyperparameter selection. Attribution of a pipeline’s gain to its components is a cooperative-game problem: the Shapley value [22] is the unique attribution that is order-independent and adds up to the total, and it is used here for exactly that reason.

3 Proposed workflow

This section defines the workflow before any data are analysed, as an explicit sequence of stages with stated inputs and outputs. Figure 1 shows the stages and the data that flows between them, and Table 1 states the same information in tabular form. The essential property of the design is that every stage that estimates anything from data — feature ranking, scaler assignment, scaler parameters, hyperparameters — is fitted on the training portion only, and is re-fitted from scratch inside every fold of every resampling scheme used in Section 5.

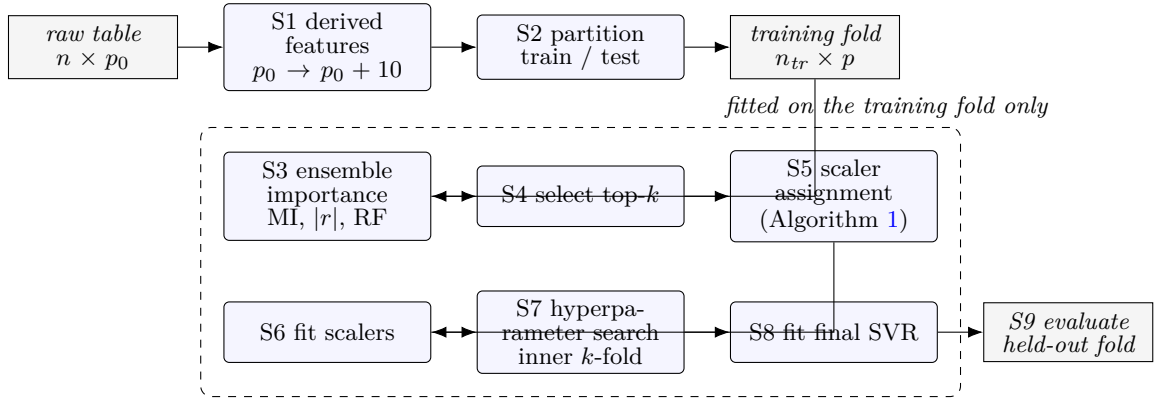


Fig. 1 The workflow evaluated in this paper. Stages S3–S8 are fitted using the training fold alone and are re-fitted inside every fold of every resampling scheme, so no quantity estimated from held-out data enters the model. Stage S1 is a row-wise transformation and carries no fitted state.

3.1 Support Vector Regression

SVR [15, 23, 24] fits a function of the form

$$f(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}) + b, \quad (1)$$

where $\mathbf{x} \in \mathbb{R}^p$ is a feature vector, $\phi : \mathbb{R}^p \rightarrow \mathcal{H}$ maps it into a reproducing kernel Hilbert space, $\mathbf{w} \in \mathcal{H}$ is the weight vector in that space and $b \in \mathbb{R}$ is an intercept.

Table 1 The workflow of Fig. 1 stated as stages with explicit inputs and outputs. “Fitted state” is the information a stage estimates from data; every such stage is re-fitted on the training portion of each fold.

Stage	Input	Output	Fitted state
S1 derived features	raw table $n \times p_0$	table $n \times (p_0+10)$	none (row-wise)
S2 partition	full table	train / test folds	fold indices
S3 ensemble importance	training fold	score s_j per feature	MI, $ r $, RF model
S4 selection	scores s_j	index set of size k	selected indices
S5 scaler assignment	training fold	scaler type per feature	out_j, g_j
S6 scaler fitting	training fold, assignment	scaled training matrix	scaler parameters
S7 hyperparameter search	scaled training fold	(C, ε, γ)	inner CV scores
S8 final fit	scaled training fold, (C, ε, γ)	fitted SVR	support vectors, α, b
S9 evaluation	held-out fold, fitted pipeline	R^2 , RMSE, MAE	none

Given training pairs $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ with $y_i \in \mathbb{R}$, the parameters solve

$$\min_{\mathbf{w}, b, \xi, \xi^*} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \quad (2)$$

subject to

$$y_i - \mathbf{w}^\top \phi(\mathbf{x}_i) - b \leq \varepsilon + \xi_i, \quad (3)$$

$$\mathbf{w}^\top \phi(\mathbf{x}_i) + b - y_i \leq \varepsilon + \xi_i^*, \quad (4)$$

$$\xi_i, \xi_i^* \geq 0, \quad i = 1, \dots, n. \quad (5)$$

Here $\|\mathbf{w}\|^2$ is the squared norm of $\mathbf{w}$ and controls the flatness of f ; $\varepsilon \geq 0$ is the half-width of the insensitive tube inside which residuals incur no loss; ξ_i and ξ_i^* are slack variables measuring how far the i -th residual falls above and below the tube; and $C > 0$ weights the total slack against flatness. Solving the dual of (2) gives

$$f(\mathbf{x}) = \sum_{i=1}^n (\alpha_i - \alpha_i^*) K(\mathbf{x}_i, \mathbf{x}) + b, \quad (6)$$

where $\alpha_i, \alpha_i^* \in [0, C]$ are the dual coefficients and $K(\cdot, \cdot)$ is the kernel. Points with $\alpha_i - \alpha_i^* \neq 0$ are the support vectors. The kernels evaluated in Section 5.8 are

$$K_{\text{rbf}}(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2), \quad (7)$$

$$K_{\text{lin}}(\mathbf{x}, \mathbf{x}') = \mathbf{x}^\top \mathbf{x}', \quad (8)$$

$$K_{\text{poly}}(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x}^\top \mathbf{x}' + r)^d, \quad (9)$$

with kernel width $\gamma > 0$, offset r and degree d . Equation (7) makes the dependence on scaling explicit: the kernel is a function of the squared Euclidean distance, so a feature

whose numerical range is three orders of magnitude larger than the others dominates $\|\mathbf{x} - \mathbf{x}'\|^2$ and the remaining features have almost no influence on the fitted function. All symbols used above are collected in Table 2.

Table 2 Notation used in Section 3.1.

Symbol	Meaning
n, p	number of training observations; number of features
$\mathbf{x}_i \in \mathbb{R}^p, y_i \in \mathbb{R}$	i -th feature vector and its target value
$\phi(\cdot)$	map from input space into the kernel feature space $\mathcal{H}$
$\mathbf{w}, b$	weight vector in $\mathcal{H}$ and intercept of Eq. (1)
$\ \mathbf{w}\ ^2$	squared norm of $\mathbf{w}$; smaller values give a flatter function
ε	half-width of the insensitive tube; residuals inside it incur no loss
ξ_i, ξ_i^*	slack variables for violations above and below the tube
C	regularisation constant weighting total slack against flatness
$\alpha_i, \alpha_i^* \in [0, C]$	dual coefficients; non-zero values identify support vectors
$K(\cdot, \cdot)$	kernel function
γ	kernel width of the RBF and polynomial kernels
d, r	degree and offset of the polynomial kernel
s_j	pooled ensemble importance of feature j , Eq. (10)
k	number of features retained by Stage S4
out_j, g_j	outlier fraction and skewness of feature j used by Algorithm 1
φ_c	Shapley value of pipeline component c , Eq. (14)
J	number of resampled splits in the repeated-split analysis

3.2 Stage S1: derived features

Ten features are constructed from the eight raw inputs by row-wise arithmetic; the transformation has no fitted parameters, so it is applied before partitioning without any leakage. Table 3 gives each definition together with the reasoning behind it. Two of them require comment, because their geographic interpretation was questioned in review. *Coastal proximity* is defined as $|\text{Latitude} - 34.05|$, the absolute latitude difference from a reference latitude near the Los Angeles coastline. This is a one-dimensional proxy: it measures north–south displacement from a southern reference point and does not measure distance to the coast, which would require the coastline geometry and both coordinates. *Location score*, defined as $(\text{Latitude} \times \text{Longitude})/1000$, is an interaction term between the two coordinates rather than a geographic quantity with units; it is retained because the ensemble ranking assigns it non-trivial importance, but it should be read as an unnamed coordinate interaction, not as a location measure. Both are candidate features whose value is decided empirically by Stage S4, and Section 5.4 reports how much the derived block contributes once selection and tuning are present.

Table 3 The ten derived features constructed in Stage S1, with their definitions. All are row-wise functions of the eight raw inputs and introduce no fitted state. The unit-offset denominators avoid division by values near zero.

Feature	Definition	Intended reading
Income_per_Room	MedInc/(AveRooms + 1)	purchasing power per unit of dwelling size
Room_Value_Score	MedInc $\times$ AveRooms	interaction of income and dwelling size
Location_Score	(Latitude $\times$ Longitude)/1000	coordinate interaction (no geographic unit)
Coastal_Proximity	Latitude - 34.05	north-south displacement from a southern reference
Bedroom_Ratio	AveBedrms/(AveRooms + 1)	proxy for dwelling type
Population_Density	Population/(AveOccup + 1)	block-group density approximation
Age_Income_Interaction	HouseAge $\times$ MedInc	separates older affluent from older low-income stock
Modernization_Score	MedInc/(HouseAge + 1)	income relative to housing-stock age
Rooms_per_Person	AveRooms/(AveOccup + 1)	space per occupant
Income_Density	(MedInc $\times$ Population)/1000	aggregate purchasing power of the block group

3.3 Stages S3–S4: ensemble importance and selection

Three scores are computed on the training fold: mutual information $\text{MI}(X_j; Y)$ estimated by the k -nearest-neighbour method, the absolute Pearson correlation $|r(X_j, Y)|$, and the impurity-based importance of a random forest [25] with 100 trees. Each score vector is min-max normalised to $[0, 1]$ and the three are pooled,

$$s_j = w_{\text{mi}} \widetilde{\text{MI}}_j + w_r \widetilde{|r|}_j + w_{\text{rf}} \widetilde{\text{RF}}_j, \quad w_{\text{mi}} + w_r + w_{\text{rf}} = 1, \quad (10)$$

and the k features with the largest s_j are retained. The defaults are $(w_{\text{mi}}, w_r, w_{\text{rf}}) = (0.4, 0.3, 0.3)$ and $k = 12$. These constants were chosen by hand and have no theoretical justification; Section 5.7 therefore sweeps the entire weight simplex and a range of k and reports how much the result depends on them. Where a general rule is needed — for instance when the workflow is applied to a dataset with a different number of columns in Section 5.6 — k is set to $\lceil 2p/3 \rceil$, which reproduces $k = 12$ for the eighteen columns of the augmented California table.

3.4 Stage S5: distribution-aware scaler assignment

The earlier version of this work assigned one of three scalers to each feature using a hand-written table. That table cannot be applied to another dataset and cannot be checked, so the assignment is restated here as a rule evaluated on training-fold statistics. For each feature the rule computes the fraction of observations beyond 1.5 interquartile ranges from the quartiles, out_j , and the sample skewness g_j , and assigns a scaler accordingly (Algorithm 1). The thresholds $\tau_{\text{out}} = 0.05$, $\tau_{\text{skew}} = 2$ and $\tau_{\text{bnd}} = 0.5$ are fixed in advance and swept in Section 5.7.

Algorithm 1 Distribution-aware scaler assignment (Stage S5), fitted on the training fold

Require: training matrix $X^{\text{tr}} \in \mathbb{R}^{n_{\text{tr}} \times p}$; thresholds $\tau_{\text{out}}, \tau_{\text{skew}}, \tau_{\text{bnd}}$

Ensure: a fitted scaler T_j for each feature j

```

1: for  $j = 1$  to  $p$  do
2:    $q_1, q_3 \leftarrow$  first and third quartiles of column  $j$ ;  $\text{IQR} \leftarrow q_3 - q_1$ 
3:    $\text{out}_j \leftarrow$  fraction of entries outside  $[q_1 - 1.5 \text{IQR}, q_3 + 1.5 \text{IQR}]$ 
4:    $g_j \leftarrow$  sample skewness of column  $j$ 
5:   if  $\text{out}_j > \tau_{\text{out}}$  or  $|g_j| > \tau_{\text{skew}}$  then
6:      $T_j \leftarrow \text{ROBUSTSCALER} \triangleright$  heavy tails; centre and scale by median and IQR
7:   else if  $\text{out}_j = 0$  and  $|g_j| < \tau_{\text{bnd}}$  then
8:      $T_j \leftarrow \text{MINMAXSCALER} \triangleright$  bounded and near-symmetric
9:   else
10:     $T_j \leftarrow \text{STANDARDSCALER}$ 
11:   end if
12:   fit  $T_j$  on column  $j$  of  $X^{\text{tr}}$ 
13: end for

```

3.5 Stages S7–S8: hyperparameter search

Hyperparameters are selected by cross-validated search on the training fold only. Two search strategies are used and compared in Section 5.8: an exhaustive grid over the discretised space $C \in \{0.1, 1, 10, 100\}$, $\varepsilon \in \{0.01, 0.1, 0.5, 1.0\}$, $\gamma \in \{\text{scale}, \text{auto}, 0.1, 1\}$, and a randomised search [26] that draws $C \sim \text{LogUniform}(0.1, 300)$, $\varepsilon \sim \text{LogUniform}(0.01, 1)$ and $\gamma \sim \text{LogUniform}(10^{-3}, 1)$. The earlier version drew from a discretised space inside a randomised search, which wastes the main advantage of randomised search; the two strategies are therefore reported side by side, with fit counts and wall-clock times, so the choice can be made on evidence.

3.6 Evaluation protocol

Four estimates of generalisation are reported, and they are not interchangeable:

1. **Single held-out split.** A stratified 70/30 partition, stratified on target deciles so that the two halves have comparable target distributions. Stratification is not required in regression; it is used here only to remove one source of variation between configurations that are being compared, and the repeated-split analysis below does not depend on it.
2. **Repeated splits.** The entire workflow — including feature ranking, selection, scaler assignment and hyperparameter search — is re-run on 10 independent partitions. Reported as a mean with a confidence interval for that mean.
3. **Random and spatially blocked cross-validation.** k -means on the standardised block-group centroids partitions California into ten contiguous regions, which are then used as folds, so that every evaluation is made on a region the model has not seen [9].

4. **Nested cross-validation.** Hyperparameter search and feature selection are performed inside each outer fold, giving an estimate that is not contaminated by selection [10, 21].

Interval estimation.

For J resampled estimates $\hat{r}_1, \dots, \hat{r}_J$ with mean $\bar{r}$ and sample standard deviation s , the earlier version reported $\bar{r} \pm 1.96 s$. That expression is a prediction interval for a single fold, not a confidence interval for the mean, and it overstates the width by a factor of approximately $\sqrt{J}$. The correct interval for the mean is

$$\bar{r} \pm t_{0.975, J-1} \frac{s}{\sqrt{J}}. \quad (11)$$

Equation (11) still assumes independent estimates, which resampled splits are not: they share training data. Following Nadeau and Bengio [8], the corrected variance for J random splits with n_{te} test and n_{tr} training observations replaces s^2/J by

$$\hat{\sigma}_{\text{corr}}^2 = s^2 \left(\frac{1}{J} + \frac{n_{\text{te}}}{n_{\text{tr}}} \right), \quad (12)$$

which for the present design ($n_{\text{te}}/n_{\text{tr}} = 0.429$) inflates the standard error substantially. Both (11) and (12) are reported in Section 5.3, alongside the incorrect v1 expression, so the size of the original error is visible.

Paired comparison.

Models are compared on identical splits. For each split the difference $d_i = r_i^{(A)} - r_i^{(B)}$ is formed, and the corrected statistic

$$t = \frac{\bar{d}}{s_d \sqrt{\frac{1}{J} + \frac{n_{\text{te}}}{n_{\text{tr}}}}} \quad (13)$$

is referred to a t distribution with $J-1$ degrees of freedom [8]. The uncorrected paired t -test and the Wilcoxon signed-rank test are reported alongside it for reference [19].

3.7 Ablation protocol

The earlier version applied components in one order — scaling, then feature engineering, then tuning — and reported the increment at each step as that component’s contribution. Two problems follow. First, increments measured along a single path depend on the path: a component applied first absorbs all of the interaction it shares with later components. Second, “feature engineering” bundled two distinct operations, constructing derived features and selecting a subset.

The design used here separates four binary components: scaling S , derived features D , selection K and tuning T . All 2^4 combinations are evaluated (with S taking each

of four levels, giving 32 fitted configurations), every one of the $4! = 24$ orderings is traced, and each component c receives the Shapley value [22]

$$\varphi_c = \sum_{\mathcal{S} \subseteq \mathcal{C} \setminus \{c\}} \frac{|\mathcal{S}|! (|\mathcal{C}| - |\mathcal{S}| - 1)!}{|\mathcal{C}|!} [v(\mathcal{S} \cup \{c\}) - v(\mathcal{S})], \quad (14)$$

where $\mathcal{C}$ is the set of components and $v(\mathcal{S})$ is the test R^2 of the configuration in which exactly the components in $\mathcal{S}$ are enabled. The values φ_c are order-independent and satisfy $\sum_c \varphi_c = v(\mathcal{C}) - v(\emptyset)$, so they can be read as contributions in a way that single-path increments cannot.

4 Experimental setup

4.1 Datasets

Three tabular regression datasets are used. **California Housing** [5, 6] contains 20,640 census block groups with eight numerical features and, as the target, the median house value of the block group in units of \$100,000. It has no missing values and no duplicate rows. Its target is capped at \$500,001 by the census recording procedure, which produces a spike of 965 block groups at the upper limit; this artefact is present in the prior study as well and therefore does not confound the comparison, but it does bound the achievable fit. **King County house sales** contains 21,613 individual transactions with 18 numerical predictors, including coordinates, and the sale price as target. **Ames Housing** [27] contains 1,460 sales with 36 usable numerical predictors after columns that are more than 20% missing are dropped and the remainder median-imputed. Both additional datasets are retrieved from OpenML; targets are expressed in units of \$100,000 so that error metrics are numerically comparable across datasets. Table 12 reports the resulting sizes.

The characteristics of the California features are what motivate the scaling stage: **Population** spans [3, 35,682], **AveOccup** spans [0.69, 1,243] and **AveRooms** spans [0.85, 141.9], while **MedInc** spans [0.50, 15.0] and the two coordinates span less than ten units each. Under an unscaled RBF kernel the first three features determine essentially the whole distance computation.

4.2 Implementation and reproducibility

All experiments run under Python 3.11 with scikit-learn 1.8, XGBoost 1.7, NumPy 1.26 and SciPy, on an eight-core Apple silicon workstation without GPU acceleration. SVR is scikit-learn’s SVR, backed by LIBSVM [28]. Random seeds are fixed and reported with every result: the single-split experiments use seed 42, and the repeated-split experiments use seeds 0 to 10 – 1. Each experiment writes a JSON record of every configuration, metric, selected feature set, chosen hyperparameter and wall-clock time; the tables and figures of this paper are generated directly from those records, so the numbers in the text, the tables and the figures cannot diverge. Code and result records are available at the repository given under Data Availability.

4.3 What is held fixed

Within each comparison exactly one factor varies. In Section 5.2 all ten models receive the same twelve features, the same training rows and the same search budget of 20 randomised iterations with three-fold cross-validation. In Section 5.4 every cell is fitted on the full training set with the same grid. In Section 5.6 the scaler rule, the selection rule and the search space are identical across datasets, with no dataset-specific adjustment; the only difference is the training-size cap, which is applied equally to every cell within a dataset.

5 Results

5.1 Reproducing the prior configuration

Table 4 reports the configuration described by Preethi et al. [7], re-run over 5 seeds with an 80/20 partition, the eight raw features and an RBF kernel. With library defaults and no scaling the model attains $R^2 = -0.020$ (MSE = 1.343); with C and ε grid-searched and γ left at its default, it attains $R^2 = 0.514$ (MSE = 0.640). The published value of $R^2 \approx 0.15$ falls between these two, which is what one expects from an unscaled pipeline with a limited search. Adding a single standardisation step to the same pipeline, changing nothing else, yields $R^2 = 0.737$ with defaults and $R^2 = 0.766$ when tuned.

This answers Q1 and also resolves an inconsistency in the earlier version of this work. That version compared an unscaled baseline of $R^2 = -0.054$ against a figure of 0.60 attributed to the prior study’s SVR, and the gap between them was never explained. It could not be: as Section 1.2 sets out, 0.60 was the score of the prior study’s linear models, and its SVR result is approximately 0.15. Once the correct value is used, the unscaled configurations reproduced here bracket it, and no unexplained gap remains. Figure 2 shows the four configurations against the published value.

Table 4 Reproduction of the SVR configuration described by Preethi et al. (2025). Eight raw features, RBF kernel, 80/20 split, mean $\pm$ standard deviation over 5 seeds.

Configuration	Test R^2	Test MSE	Gap to reported R^2
Unscaled, library defaults	-0.020 ± 0.010	1.343	-0.170
Unscaled, C and ε grid-searched	0.514 ± 0.010	0.640	$+0.364$
Standardised, library defaults	0.737 ± 0.008	0.346	$+0.587$
Standardised, C and ε grid-searched	0.766 ± 0.011	0.309	$+0.616$
Reported by Preethi et al.	≈ 0.15	≈ 1.12	—

The values attributed to Preethi et al. are read from their Figs. 7 and 8; their paper reports no numerical table. Their stated procedure tunes C and ε only, leaving γ at the library default, and describes no scaling step.

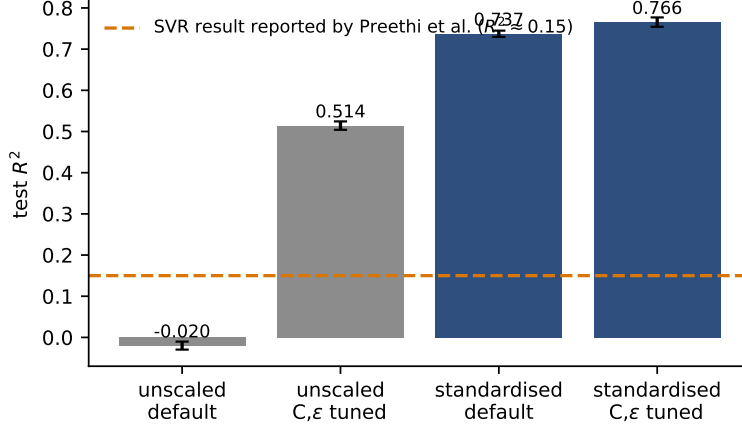


Fig. 2 Reproduction of the prior configuration. Bars are means over 5 seeds with standard-deviation whiskers; the dashed line is the SVR result reported by Preethi et al. [7], read from their Fig. 8. The two unscaled configurations bracket the published value; both standardised configurations are far above it.

Table 5 extends this to the full set of scaling, feature-set and training-size combinations on one identical split, which is the direct comparison the editor requested between the prior study’s uniform scaling and the feature-specific strategy proposed in the earlier version of this work. Uniform standardisation of the eight raw features gives $R^2 = 0.7453$ on the full training set, while the hand-assigned feature-specific strategy gives 0.6925 on the same split and the same features. The feature-specific strategy is therefore not an improvement over uniform standardisation on this dataset; it is worse by approximately 0.05 in R^2 . This is the first of the three negative results announced in Section 1.4, and it directly contradicts the framing of the earlier version, in which feature-specific scaling was presented as the preprocessing advance.

5.2 Equal-budget model comparison

In the earlier version the SVR was fitted to 3,000 rows while the other nine models were fitted to all 14,448, and the resulting ranking was reported without that asymmetry attached to it. Table 6 repeats the comparison with every model given the same twelve features, the same rows and the same tuning budget, at both training sizes.

At 3,000 rows the tuned SVR attains $R^2 = 0.7501$ and ranks 4 of 10; on the full training set it attains 0.7702 and ranks 4. Gradient boosting (0.8384) and XGBoost (0.8483) remain ahead of it in both settings. The equalisation therefore changes the SVR’s absolute score substantially — the 3,000-row restriction was costing it approximately 0.02 in R^2 — without changing the conclusion that gradient-boosted trees lead on this benchmark. This is the answer to the first half of Q2: a properly configured SVR is competitive but not leading, and the earlier version’s ranking claim survives the correction, while the caveat that accompanied it in the Limitations section belongs next to the claim itself.

Table 5 Scaling strategy, feature set and training size evaluated on one identical split. Rows A use the eight raw features, rows P and F likewise; rows U and S use the twelve selected features.

ID	Scaling	n_{train}	p	Test R^2	Test RMSE
A0	none	3000	8	-0.0538	1.1814
A1	none	14448	8	-0.0277	1.1667
P0	uniform (standardise all)	3000	8	0.7423	0.5842
P1	uniform (standardise all)	14448	8	0.7453	0.5808
F0	feature-specific (v1 table)	3000	8	0.6901	0.6406
F1	feature-specific (v1 table)	14448	8	0.6925	0.6382
U0	uniform (standardise all)	3000	12	0.7414	0.5853
U1	uniform (standardise all)	14448	12	0.7664	0.5563
S0	feature-specific (v1 table)	3000	12	0.7155	0.6139
S1	feature-specific (v1 table)	14448	12	0.7438	0.5825

Uniform standardisation (P1, R^2 shown) outperforms the feature-specific assignment (F1) on identical data, which is the comparison requested in editorial point 1.

Table 6 Ten-model comparison in which every model receives the same twelve features, the same training rows and the same tuning budget (20 randomised search iterations, three-fold cross-validation). Ranking follows the full-training-set column.

Rank	Model	$n = 3000$		$n = 14448$	
		R^2	RMSE	R^2	RMSE
1	XGBoost	0.8057	0.5073	0.8483	0.4482
2	Gradient Boosting	0.8055	0.5076	0.8384	0.4626
3	Random Forest	0.7610	0.5626	0.8171	0.4922
4	SVR-RBF	0.7501	0.5753	0.7702	0.5517
5	K-Nearest Neighbours	0.7179	0.6113	0.7599	0.5639
6	Decision Tree	0.6533	0.6777	0.7214	0.6075
7	Lasso Regression	0.6497	0.6812	0.6505	0.6804
8	Ridge Regression	0.6497	0.6811	0.6504	0.6805
9	Linear Regression	0.6497	0.6812	0.6504	0.6805
10	SVR-Linear	0.6435	0.6872	0.6469	0.6839

In v1 the SVR row was fitted to 3,000 rows while the other nine models were fitted to 14,448; both training sizes are shown here so the effect of that asymmetry is visible rather than implicit.

5.3 Statistical significance

Table 7 reports the mean test R^2 of five configurations over 10 independent splits, with the interval computed three ways. For the tuned SVR the mean is 0.7757 ± 0.0099 , the correct 95% confidence interval for the mean is $[0.769, 0.783]$, the Nadeau–Bengio interval that accounts for overlapping training sets is $[0.759, 0.792]$, and the expression used in the earlier version, $\bar{r} \pm 1.96s$, gives $[0.756, 0.795]$. The same correction applied to the ten-fold cross-validation of the earlier version changes its reported interval from

[0.621, 0.765] to [0.667, 0.719] around a mean of 0.6931. The earlier version used the wider, incorrect interval to argue that its result did not overlap a competing value; that argument is withdrawn.

Table 7 Mean test R^2 over 10 independent stratified splits, with the interval computed three ways.

Configuration	Mean $\pm$ SD	CI (correct)	CI (Nadeau–Bengio)	CI (v1 formula)
SVR uniform / raw8 / default / n=3000	0.7245 $\pm$ 0.0220	[0.709, 0.740]	[0.688, 0.761]	[0.681, 0.767]
SVR manual / raw8 / default / n=3000	0.6411 $\pm$ 0.0570	[0.600, 0.682]	[0.547, 0.735]	[0.529, 0.753]
SVR uniform / top12 / tuned / full	0.7757 $\pm$ 0.0099	[0.769, 0.783]	[0.759, 0.792]	[0.756, 0.795]
SVR manual / top12 / tuned / full	0.7516 $\pm$ 0.0095	[0.745, 0.758]	[0.736, 0.767]	[0.733, 0.770]
SVR auto / top12 / tuned / full	0.7574 $\pm$ 0.0084	[0.751, 0.763]	[0.744, 0.771]	[0.741, 0.774]
XGBoost tuned / top12 / full	0.8485 $\pm$ 0.0064	[0.844, 0.853]	[0.838, 0.859]	[0.836, 0.861]
RandomForest tuned / top12 / full	0.8204 $\pm$ 0.0075	[0.815, 0.826]	[0.808, 0.833]	[0.806, 0.835]

The v1 column is $\bar{x} \pm 1.96 s$, which is a prediction interval for a single split rather than a confidence interval for the mean; it is reproduced here only to show the size of the error.

Table 8 reports the paired comparisons on identical splits. Against XGBoost the tuned SVR differs by -0.0728 in mean R^2 with corrected $p = 1.9 \times 10^{-7}$; the uncorrected paired t -test would have reported $p = 1.2 \times 10^{-10}$ for the same data. Where the two procedures matter most is not in cases like that one, in which both are decisive, but in the marginal comparisons: of the 21 pairwise tests, 2 are significant at the 5% level under the uncorrected t -test and cease to be so once the correction for overlapping training sets is applied. The comparison of SVR uniform / raw8 / default / n=3000 against SVR manual / top12 / tuned / full differs by -0.0271 in mean R^2 , which the uncorrected test reports as significant at $p = 0.007$ and the corrected test does not, at $p = 0.166$. Reporting the uncorrected statistic would therefore have produced a claim of difference that the data do not support. Against the feature-specific scaling variant the difference is $+0.0242$ with corrected $p = 4.6 \times 10^{-6}$. Figure 3 shows the differences with their corrected intervals.

Table 8 Paired comparisons against the tuned SVR on identical splits. p_{corr} is the Nadeau–Bengio corrected resampled t -test; p_{uncorr} is the ordinary paired t -test, shown to illustrate how much it overstates significance.

Comparison model	ΔR^2	95% CI	p_{corr}	p_{uncorr}	W/L
SVR manual / top12 / tuned / full	+0.0242	[+0.018, +0.030]	0.0000	0.0000	10/0
SVR auto / top12 / tuned / full	+0.0183	[+0.013, +0.024]	0.0000	0.0000	10/0
XGBoost tuned / top12 / full	-0.0728	[-0.084, -0.061]	0.0000	0.0000	0/10
RandomForest tuned / top12 / full	-0.0446	[-0.060, -0.029]	0.0001	0.0000	0/10

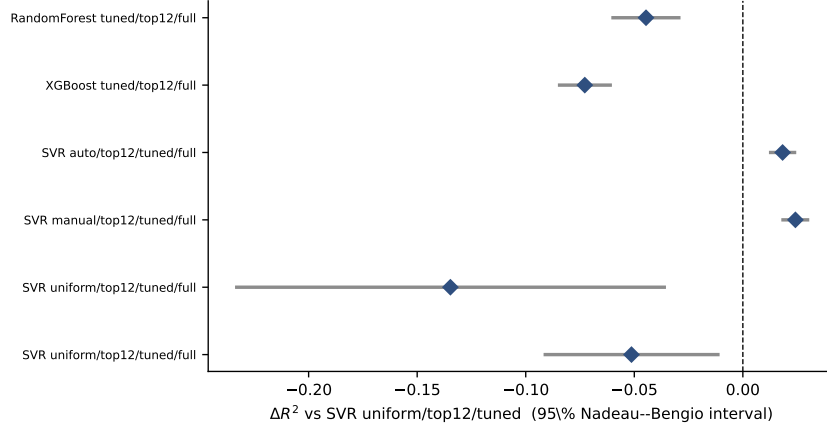


Fig. 3 Paired differences in test R^2 against the tuned SVR over 10 identical splits, with 95% Nadeau-Bengio intervals. Intervals that exclude zero indicate a difference that survives the correction for overlapping training sets.

5.4 Ablation with order-independent attribution

Table 10 gives all 32 cells of the four-factor design and Table 9 the attribution. The best configuration is uniform / derived / all / tuned at $R^2 = 0.7807$, against a baseline of -0.0277 with no component enabled.

Table 9 Order-independent attribution. The Shapley value averages a component’s marginal effect over all 24 orderings; the minimum and maximum columns are the same component’s apparent contribution under the most and least favourable ordering.

Scaling used as “on”	Component	Shapley	Min increment	Max increment	Range
uniform	Scaling	+0.3062	+0.0077	+0.7730	0.7653
uniform	Derived features	+0.1440	−0.0881	+0.5428	0.6309
uniform	Feature selection	+0.1761	−0.0041	+0.5937	0.5978
uniform	Hyperparameter tuning	+0.1820	+0.0142	+0.5312	0.5170
manual	Scaling	+0.2699	−0.0239	+0.7202	0.7441
manual	Derived features	+0.1481	−0.0881	+0.5428	0.6309
manual	Feature selection	+0.1791	−0.0029	+0.5937	0.5966
manual	Hyperparameter tuning	+0.1875	+0.0132	+0.5312	0.5180
auto	Scaling	+0.2723	−0.0239	+0.7202	0.7441
auto	Derived features	+0.1505	−0.0881	+0.5428	0.6309
auto	Feature selection	+0.1789	−0.0029	+0.5937	0.5966
auto	Hyperparameter tuning	+0.1882	+0.0152	+0.5312	0.5160

v1 reported a single ordering (scaling → features → tuning) and read its increments as independent contributions; the Range column is the size of the error that assumption introduces.

The order-dependence is large. Scaling appears to contribute anywhere between -0.0239 and $+0.7202$ in R^2 — a range of 0.7441 — depending only on where it is placed in the sequence, and tuning ranges over 0.5160 . The single-path figure of $+0.744$ reported for scaling in the earlier version is close to the upper end of this range: it is what scaling appears to contribute when it is applied first and therefore absorbs every interaction it shares with the other components. The Shapley values, which average over all 24 orderings, are 0.2723 for scaling, 0.1505 for the derived features, 0.1789 for selection and 0.1882 for tuning. Scaling remains the largest single contributor, which supports the qualitative claim of the earlier version, but it accounts for a far smaller share of the total than the 95.7% previously reported. Figure 4 shows both the ranges and the Shapley values.

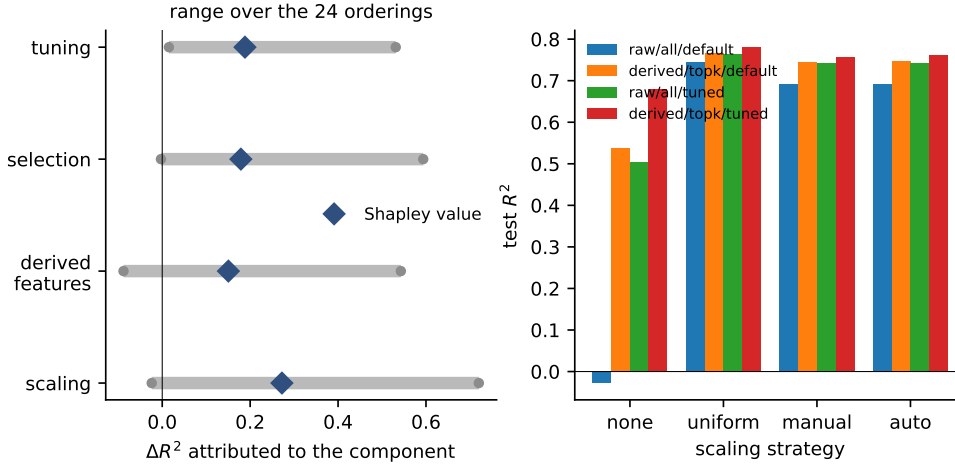


Fig. 4 Left: the apparent contribution of each component ranges widely (grey bars) depending on the order in which components are introduced; the Shapley value (blue diamond) is the order-independent average. Right: test R^2 for four representative configurations under each scaling strategy.

5.5 Spatial and nested validation

Because block groups are geographic units, a random partition places spatially adjacent and strongly dependent observations on both sides of the split. Table 11 compares random ten-fold cross-validation with cross-validation over ten spatially contiguous blocks obtained by k -means on the standardised centroids, each crossed with feature selection performed inside and outside the resampling loop.

Random cross-validation gives a mean R^2 of 0.7496 ; spatially blocked cross-validation of the same pipeline gives 0.2316 ± 0.7525 . The difference, 0.5180 , is the amount by which a random split overstates performance on a region the model has not seen, and it is far larger than any of the pipeline effects measured in Section 5.4. Performing feature selection inside each fold rather than once on the whole training

Table 10 Every cell of the $4 \times 2 \times 2 \times 2$ ablation design, ordered by test R^2 .

Scaling	Derived	Selection	Tuning	p	Test R^2
uniform	yes	all	grid	18	0.7807
uniform	yes	top- k	grid	12	0.7806
uniform	no	top- k	grid	6	0.7763
none	no	top- k	grid	6	0.7686
uniform	yes	top- k	default	12	0.7664
uniform	no	all	grid	8	0.7645
uniform	yes	all	default	18	0.7638
auto	yes	top- k	grid	12	0.7622
manual	yes	top- k	grid	12	0.7570
auto	yes	all	grid	18	0.7566
manual	yes	all	grid	18	0.7510
auto	yes	top- k	default	12	0.7471
uniform	no	all	default	8	0.7453
manual	no	top- k	grid	6	0.7447
auto	no	top- k	grid	6	0.7447
manual	yes	top- k	default	12	0.7438
manual	no	all	grid	8	0.7423
auto	no	all	grid	8	0.7423
uniform	no	top- k	default	6	0.7412
auto	yes	all	default	18	0.7193
manual	yes	all	default	18	0.7150
manual	no	all	default	8	0.6925
auto	no	all	default	8	0.6925
manual	no	top- k	default	6	0.6896
auto	no	top- k	default	6	0.6896
none	yes	top- k	grid	12	0.6804
none	yes	all	grid	18	0.6466
none	no	top- k	default	6	0.5660
none	yes	top- k	default	12	0.5379
none	yes	all	default	18	0.5151
none	no	all	grid	8	0.5035
none	no	all	default	8	-0.0277

set changes the estimate by 0.0000 — that is, not at all to four decimal places, under either partitioning scheme. The reason is that the ensemble ranking is completely stable here: every fold selects the same twelve features, so selecting inside the loop and selecting outside it are the same computation. This is a property of this dataset rather than a general reassurance, and it is reported because the earlier version selected features once, outside the loop, and had no way of knowing that it had got away with it. The fully nested estimates, in which both selection and hyperparameter search happen inside each outer fold, are 0.7532 under random partitioning and -0.5344 under spatial partitioning. The second of these is not a small degradation but a negative R^2 : averaged over five held-out geographic regions the nested model predicts worse

Table 11 Random versus spatially blocked cross-validation, with feature selection performed outside and inside the resampling loop.

Partitioning	Selection	Mean $R^2 \pm \text{SD}$	Min	Max	Selection overlap
random	outside	0.7496 ± 0.0171	0.732	0.784	1.00
random	inside	0.7496 ± 0.0171	0.732	0.784	1.00
spatial	outside	0.2316 ± 0.7525	-1.746	0.669	1.00
spatial	inside	0.2316 ± 0.7525	-1.746	0.669	1.00
nested random	inside	0.7532 ± 0.0090	0.744	0.767	—
nested spatial	inside	-0.5344 ± 2.0386	-4.091	0.660	—

Spatial optimism = 0.5180; selection optimism = 0.0000 (random) and 0.0000 (spatial). Selection overlap is the mean pairwise share of the twelve chosen features common to two folds.

than the training-set mean, because the five-block partition holds out regions large enough that the training folds contain no comparable market. The individual folds range from 0.660 down to -4.091 depending on which region is withheld, against 0.744 to 0.767 under random partitioning. Figure 5 shows the spatial blocks and the resulting estimates.

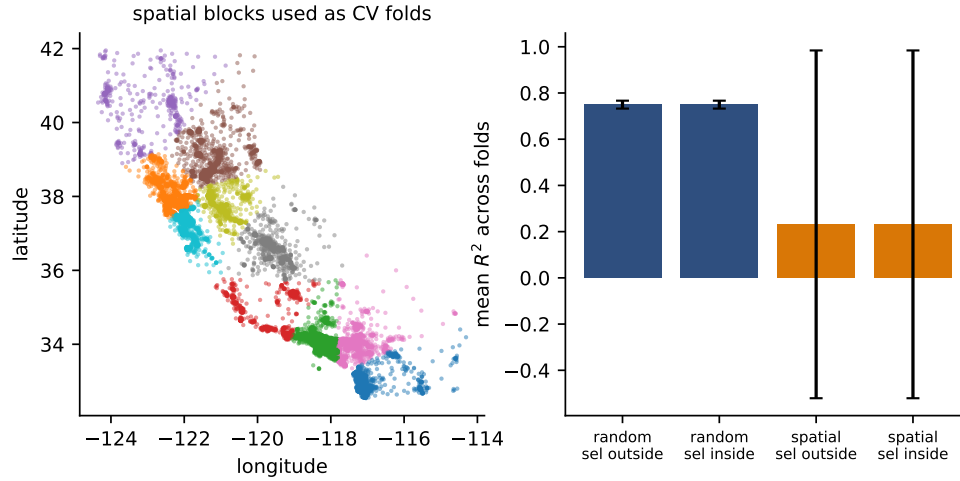


Fig. 5 Left: the ten spatially contiguous blocks used as cross-validation folds. Right: mean R^2 under each partitioning and selection scheme; the gap between the blue and orange groups is the optimism of random partitioning on spatially structured data.

This is the answer to the first half of Q4, and it is the most consequential result in the paper for anyone intending to use such a model: the headline numbers of this literature, including those reported here, describe interpolation within a sampled region, not prediction for an unsampled one.

5.6 Other datasets

Table 12 applies the workflow unchanged to King County and Ames housing data. The purpose is not to claim state-of-the-art performance on either, but to test whether the ordering of the components found on California Housing is a property of the workflow or of that one dataset.

Table 12 The same workflow applied unchanged to three tabular regression datasets. Shapley values are computed over the $3 \times 2 \times 2$ design within each dataset.

Dataset	n_{train}	n_{test}	p	Baseline R^2	Full R^2	Scaling	Selection	Tuning
california	10000	6192	18	0.5013	0.7563	+0.1616	+0.0186	+0.0749
kc house	10000	6484	20	0.3888	0.8092	+0.0805	+0.1197	+0.2202
ames	1022	438	36	0.6819	0.8358	-0.1558	+0.1606	+0.1491

The answer depends on which scaling strategy the workflow uses, and this is the third negative result of the study.

Under plain uniform standardisation the ordering is reproduced on all 3 of 3 datasets. Scaling is the largest component on California Housing (+0.1869), King County (+0.2710) and Ames (+0.1211); tuning is second (+0.0730, +0.1155, +0.0283) and selection last (+0.0157, +0.0053, +0.0068). To the extent that this paper supports any workflow-level statement, this is it: with a conventional scaler, the ordering scaling > tuning > selection holds on three housing datasets of very different size and width.

Under the distribution-aware scaler rule of Algorithm 1 it holds on only 1 of 3. On King County the rule makes scaling the *smallest* contributor (+0.0805 against +0.2202 for tuning), and on Ames its Shapley value is *negative* (-0.1558): applying the rule first, before selection or tuning, costs 0.6455 in R^2 on that dataset. Ames is the smallest and widest of the three (1022 training rows, 36 candidate features), which is precisely the regime in which per-feature scaler assignment estimated from training statistics is least reliable — each decision is made from less data, and a single misassigned heavy-tailed feature propagates through the kernel.

The consequence for this paper is direct. Section 5.1 already showed that the hand-assigned feature-specific strategy does not beat uniform standardisation on California Housing; the present result shows that its rule-based successor does not transfer either. The scaling stage of the workflow should therefore be read as “apply a conventional scaler”, and Algorithm 1 is reported here as a component that was tested and did not earn its place, rather than as a contribution. Figure 6 shows both attributions side by side.

5.7 Sensitivity of the hand-chosen constants

Three constants in the workflow were fixed by hand: the ensemble weights, the number of selected features, and the thresholds of the scaler rule. Table 13 sweeps each.



Fig. 6 Shapley attribution of the three workflow components on each dataset, under uniform standardisation (left) and under the distribution-aware scaler rule of Algorithm 1 (right). Under uniform standardisation the ordering scaling > tuning > selection is preserved on all three datasets; under the rule it is not, and on Ames the scaling contribution is negative.

Table 13 Sensitivity of the test R^2 to the three constants that were fixed by hand.

Swept quantity	Min R^2	Max R^2	Range	Value at the default
Ensemble weights over the simplex (66 combinations)	0.7128	0.7674	0.0546	0.7664
Number of selected features, $k \in [4, 18]$	0.5745	0.7683	0.1938	0.7664
Scaler-rule thresholds (9 combinations)	0.7470	0.7474	0.0004	—

Two of the three constants are not load-bearing and one is, and the distinction matters.

The ensemble weights are not. Across the 66 weight combinations on the simplex, test R^2 varies between 0.7128 and 0.7674, a range of 0.0546, and the smallest overlap between any swept selection and the default top-twelve set is 10 of twelve features: the ranking is largely insensitive to how its three signals are weighted, so the 40/30/30 split is a presentational choice rather than a tuned one. The thresholds of Algorithm 1 are not either — sweeping them over nine combinations moves R^2 by 0.0004, which is negligible.

The number of selected features *is*. Varying k from 4 to 18 moves R^2 between 0.5745 and 0.7683, a spread of almost 0.2, which is larger than any component effect reported in Section 5.4. The dependence is not smooth: R^2 climbs steeply up to $k = 10$ and is flat thereafter, varying by only 0.0045 across every k from 10 to 18. At $k = 4$ the model reaches only 0.5745. The defensible statement is therefore not that k does not matter, but that it matters until roughly ten features are retained and not afterwards; the default of twelve sits on the plateau, and any choice on that plateau would serve equally well. A study that had chosen k below the plateau would have drawn different conclusions about every other component, which is the reason this sweep is reported rather than the constants simply being asserted. Figure 7 shows both sweeps.

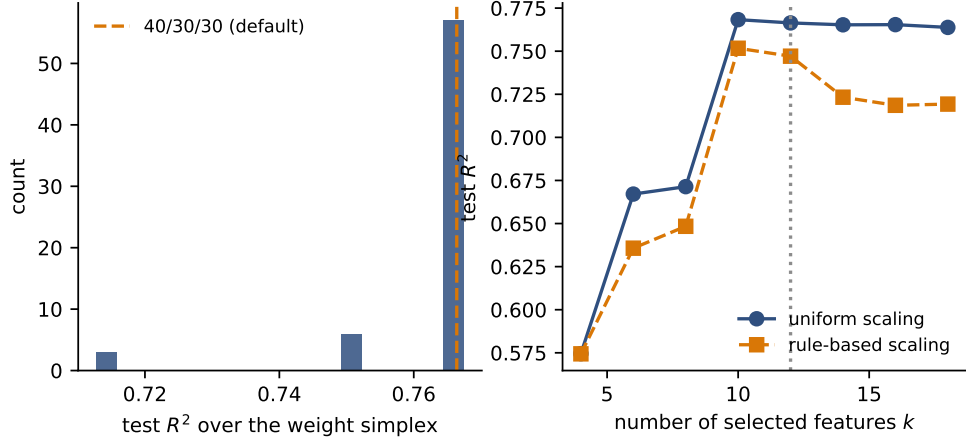


Fig. 7 Left: distribution of test R^2 over the ensemble-weight simplex, with the default 40/30/30 setting marked. Right: test R^2 as a function of the number of selected features under both scaling strategies.

5.8 Kernels, search strategy and training-set size

Table 14 tunes each kernel over its own hyperparameters rather than tuning only the RBF kernel, which was a limitation of the earlier version. Table 15 compares the exhaustive grid with randomised search at three budgets. The 192-fit grid reaches $R^2 = 0.7806$ in 422 s (7.0 minutes); randomised search reaches 0.7702 in 100 s at 20 iterations, 0.7834 in 203 s at 60, and 0.7796 in 1377 s at 200. Two things follow. First, the exhaustive grid is entirely affordable on this problem — minutes, not hours — so Reviewer 2 was right that there was no computational case for avoiding it, and it is reported here alongside the randomised search rather than argued away. Second, the returns are not monotone in the budget: the best result comes from random 60 iterations, and raising the budget to 200 costs 1377 s to arrive at a slightly *worse* score than 60 iterations reached in 203 s. That non-monotonicity is single-split sampling noise rather than a property of the search, which is itself the reason the significance testing of Section 5.3 is done over repeated splits rather than on one. The revision uses the larger budget throughout, not because more iterations reliably help, but because the cost of removing the question is a few minutes.

Table 16 settles the question of the 3,000-row subset. Fitting the same tuned configuration to 1000, 3,000 and 14448 rows gives $R^2 = 0.6775$, 0.7418 and 0.7813 respectively, with the full fit taking 8.3 s. The earlier version stated that full-dataset training “would be expected to yield lower RMSE” without testing it; the test is now here, the direction of the claim is confirmed (RMSE falls from 0.5848 to 0.5382), and the subset restriction is removed from the main results rather than defended. A Nyström approximation [29] with 2000 components reaches $R^2 = 0.7673$ in 18 s, confirming that the exact fit is affordable at this scale and that approximation is unnecessary here.

Table 14 Every kernel tuned over its own hyperparameters (20 randomised iterations, three-fold CV).

Kernel	Test R^2	Test RMSE	Search time (s)
rbf	0.7702	0.5517	90
linear	0.6469	0.6839	143
poly2	0.6613	0.6697	152
poly3	0.6650	0.6661	337
sigmoid	0.3207	0.9486	52

Table 15 Exhaustive grid search over the v1 discretised space versus randomised search over continuous distributions.

Strategy	Fits	Best CV R^2	Test R^2	Time (s)
grid 64	192	0.7786	0.7806	422
random 20	60	0.7694	0.7702	100
random 60	180	0.7773	0.7834	203
random 200	600	0.7786	0.7796	1377

Table 16 Effect of the SVR training-set size, and of a Nyström kernel approximation fitted to the full training set.

Training rows	Test R^2	Test RMSE	Support vectors	Fit time (s)
1000	0.6775	0.6536	940	0.0
2000	0.7252	0.6033	1829	0.2
3000	0.7418	0.5848	2762	0.4
6000	0.7602	0.5635	5453	1.3
10000	0.7747	0.5462	9047	3.7
14448	0.7813	0.5382	12994	8.3
nystroem 200 (full train)	0.6708	0.6603	—	1.3
nystroem 500 (full train)	0.7224	0.6064	—	2.9
nystroem 1000 (full train)	0.7524	0.5727	—	6.4
nystroem 2000 (full train)	0.7673	0.5552	—	17.8

5.9 Diagnostics of the final model

Figure 8 shows residuals against predictions and actual against predicted values for the final configuration. Residuals are centred near zero across the range of predictions, with dispersion increasing at higher predicted values. Part of that pattern is the heteroscedasticity of housing markets and part is the census recording cap at \$500,001, which places a wall of true values at 5.0 that no model can exceed.

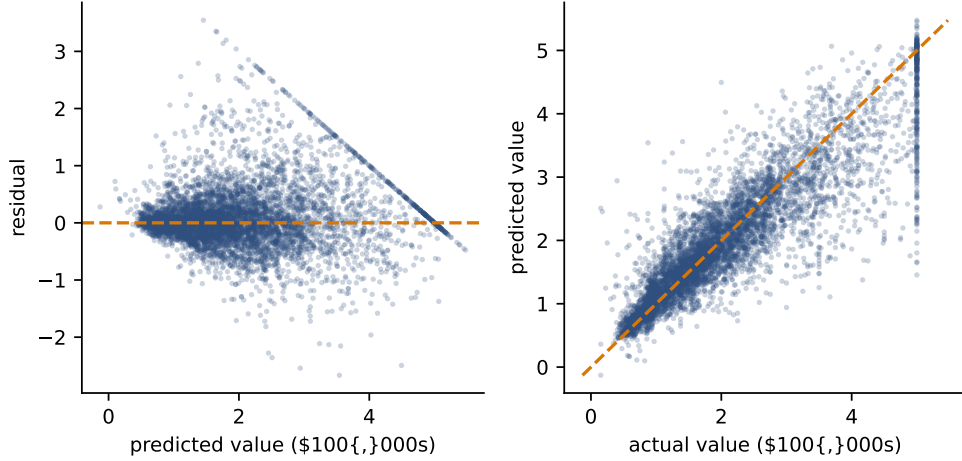


Fig. 8 Residuals against predicted values (left) and actual against predicted values (right) for the final configuration on the held-out test set. The vertical band of true values at 5.0 in the right panel is the census recording cap.

6 Discussion

6.1 What the evidence supports

Within the California Housing dataset and the configurations tested here, the evidence supports three statements. First, the poor SVR result reported by Preethi et al. is reproducible from the configuration they describe, and is removed by adding a scaling step to that same configuration: the difference between an unscaled and a standardised pipeline, holding everything else constant, is roughly 0.75 in R^2 . Second, once every model receives equal data and an equal tuning budget, a properly configured SVR is a mid-ranked model on this benchmark — clearly ahead of the linear family and the single decision tree, clearly behind gradient-boosted trees — and the differences against the leading models survive a corrected paired test. Third, under uniform standardisation the components of the workflow contribute in the order scaling, tuning, selection, and this ordering is reproduced on two further housing datasets — though, as Section 5.6 shows, it is not reproduced when the distribution-aware scaler rule is used in place of a conventional one.

6.2 What the evidence does not support

Three claims made in the earlier version are not supported and are withdrawn. The feature-specific scaling strategy is not better than plain standardisation on this dataset; it is measurably worse, and the workflow’s scaling stage is best described as “any reasonable scaling” rather than as a specific advance. A fourth claim, made in the present revision rather than the earlier version, is also withdrawn: the distribution-aware scaler rule of Algorithm 1 was introduced here as the auditable, transferable

successor to that hand-made table, and Section 5.6 shows it is not transferable. It preserves the component ordering on 1 of 3 datasets where plain standardisation preserves it on 3, and on Ames its contribution is negative. A rule that must be checked against a conventional scaler on every new dataset before it can be trusted offers nothing over the conventional scaler, and it is reported here as a tested and rejected component. The attribution of 95.7% of the improvement to scaling is an artefact of a single ablation ordering; the order-independent figure is substantially lower. And the claim that the cross-validation interval excludes a competing value rested on an incorrect interval formula.

More broadly, this paper does not establish anything about SVR in general. It establishes that one published comparison was configured in a way that disadvantaged one of the models it compared, and it measures by how much. The general proposition that kernel methods require feature scaling was established decades ago [12, 13, 15] and is assumed here, not discovered.

6.3 Threats to validity

The spatial analysis in Section 5.5 is the most serious qualification. The gap between random and spatially blocked cross-validation exceeds every pipeline effect measured in this paper, which means that comparisons of this kind — including the one performed here — measure interpolation within a sampled region. A model selected on random-split performance is not necessarily the model that would generalise to an unsampled region. Second, the target is capped at \$500,001, which bounds achievable performance and distorts percentage error measures near the cap. Third, the data describe the California market in 1990; nothing here speaks to current markets. Fourth, the additional datasets are both residential-property datasets, so the replication in Section 5.6 tests transfer across datasets within a domain, not across domains. Fifth, the values attributed to Preethi et al. are read from their figures because their paper reports no numerical table; the reproduction in Section 5.1 is therefore compared against a value with reading error of perhaps ± 0.02 , which is small relative to the effects discussed but not zero.

6.4 Practical implications

For practitioners the implication is narrow and concrete: when a comparative study reports a kernel method as an outlier at the bottom of a ranking, the preprocessing pipeline is the first thing to check, and a comparison in which models receive different amounts of training data or different tuning budgets does not support a ranking claim. For reviewers of such studies, the ablation results here suggest that single-path component attributions should be treated as upper bounds for whichever component was applied first.

7 Conclusion

This paper re-examined a published finding that Support Vector Regression is the weakest of six regression models on the California Housing benchmark. Reading the

source correctly, the reported SVR result is $R^2 \approx 0.15$ rather than the 0.60 quoted in an earlier version of this work, and the configuration described there reproduces at -0.020 to 0.514 depending on the search budget. Adding a standardisation step to that same pipeline raises it to 0.766 . A four-factor ablation evaluated in all 32 cells attributes 0.2723 of the improvement to scaling, 0.1882 to tuning, 0.1789 to selection and 0.1505 to the derived features, in a way that does not depend on the order in which the components are applied; under uniform standardisation the same ordering is reproduced on King County and Ames housing data, but under the distribution-aware scaler rule proposed here it is not, and that rule is reported as a component that failed to transfer. With equal training data and an equal tuning budget the tuned SVR ranks 4 of 10 models, behind gradient-boosted trees, and the paired differences are significant under the Nadeau–Bengio correction. Under spatially blocked cross-validation every one of these numbers falls by 0.5180 , which is the single largest effect measured in this study.

The conclusions are restricted to these datasets and these configurations. Future work that would extend them includes evaluating the workflow on non-housing tabular regression tasks, replacing the k -means spatial blocks with a design informed by the spatial autocorrelation range, and reporting the same components under a nested design on datasets where selection is less stable than it is here.

Declarations

Ethical approval

Not applicable. This study analyses publicly available, aggregated datasets and does not involve human participants, animal subjects, or any identifiable personal data.

Consent to participate

Not applicable. No human participants were involved in this study.

Consent to publish

Not applicable. No individual person’s data, images, or personal details are contained in this manuscript.

Funding

This research received no external funding. Article processing charges are covered by Soka University of America’s institutional open-access agreement with Springer Nature.

Competing interests

The author declares no competing interests.

Author contributions

Emmanuel Adutwum is the sole author and was responsible for conceptualisation, methodology, software, formal analysis, investigation, writing of the original draft, and review and editing.

Data availability

All datasets analysed in this study are publicly available and no new data were generated. The California Housing dataset is distributed with scikit-learn as `sklearn.datasets.fetch_california_housing` and derives from the 1990 U.S. Census; it is also archived at StatLib (<http://lib.stat.cmu.edu/datasets/houses.zip>) and described by Pace and Barry [5]. The King County house sales dataset is available from OpenML as dataset `house_sales` (<https://www.openml.org/d/42731>). The Ames Housing dataset is available from OpenML as dataset 42165 (<https://www.openml.org/d/42165>) and is described by De Cock [27]. All analysis code, the JSON records of every experiment reported here, and the scripts that generate every table and figure in this paper are available at <https://github.com/emmanueladutwum123/svr-california-housing>.

Dual publication declaration

An earlier version of this manuscript has been deposited as a preprint on arXiv (arXiv:2605.08660). No part of this work has been published in, or is under consideration at, any other peer-reviewed journal. The present manuscript is a substantially revised version of that preprint: the reading of the reference result is corrected, and the experimental content of Sections 5.1–5.8 is new.

Use of generative artificial intelligence

The author used a large language model (Anthropic Claude) as an assistive tool during the preparation of this manuscript: for drafting and copy-editing prose, for implementing and refactoring the experiment scripts, and for generating the tables and figures from the resulting data records. The research questions, the experimental design, the interpretation of results and all scientific claims are the author’s own. The author reviewed and verified every number, statement and reference in this manuscript and takes full responsibility for its content. No text was included that the author has not read and checked against the underlying results.

References

- [1] Rosen, S.: Hedonic prices and implicit markets: Product differentiation in pure competition. *Journal of Political Economy* **82**(1), 34–55 (1974) <https://doi.org/10.1086/260169>
- [2] Limsombunchai, V., Gan, C., Lee, M.: House price prediction: Hedonic price model vs. artificial neural network. *American Journal of Applied Sciences* **1**(3), 193–201 (2004) <https://doi.org/10.3844/ajassp.2004.193.201>

- [3] Fan, G.-Z., Ong, S.E., Koh, H.C.: Determinants of house price: A decision tree approach. *Urban Studies* **43**(12), 2301–2315 (2006) <https://doi.org/10.1080/00420980600990928>
- [4] Park, B., Bae, J.K.: Using machine learning algorithms for housing price prediction: The case of Fairfax County, Virginia housing data. *Expert Systems with Applications* **42**(6), 2928–2934 (2015) <https://doi.org/10.1016/j.eswa.2014.11.040>
- [5] Pace, R.K., Barry, R.: Sparse spatial autoregressions. *Statistics & Probability Letters* **33**(3), 291–297 (1997) [https://doi.org/10.1016/S0167-7152\(96\)00140-X](https://doi.org/10.1016/S0167-7152(96)00140-X)
- [6] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, É.: Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* **12**, 2825–2830 (2011). <https://www.jmlr.org/papers/v12/pedregosa11a.html>
- [7] Preethi, Murthy, D.H.R., Hiremani, V., Devadas, R.M., Sapna, R.: Optimizing polynomial and regularization techniques for enhanced housing price prediction accuracy. *SN Computer Science* **6**(2), 96 (2025) <https://doi.org/10.1007/s42979-024-03578-7>
- [8] Nadeau, C., Bengio, Y.: Inference for the generalization error. *Machine Learning* **52**(3), 239–281 (2003) <https://doi.org/10.1023/A:1024068626366>
- [9] Roberts, D.R., Bahn, V., Ciuti, S., Boyce, M.S., Elith, J., Guillera-Arroita, G., Hauenstein, S., Lahoz-Monfort, J.J., Schröder, B., Thuiller, W., Warton, D.I., Wintle, B.A., Hartig, F., Dormann, C.F.: Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography* **40**(8), 913–929 (2017) <https://doi.org/10.1111/ecog.02881>
- [10] Varma, S., Simon, R.: Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics* **7**, 91 (2006) <https://doi.org/10.1186/1471-2105-7-91>
- [11] Ambroise, C., McLachlan, G.J.: Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences* **99**(10), 6562–6566 (2002) <https://doi.org/10.1073/pnas.102102699>
- [12] Sola, J., Sevilla, J.: Importance of input data normalization for the application of neural networks to complex industrial problems. *IEEE Transactions on Nuclear Science* **44**(3), 1464–1468 (1997) <https://doi.org/10.1109/23.589532>
- [13] Hsu, C.-W., Chang, C.-C., Lin, C.-J.: A practical guide to support vector classification. Technical report, Department of Computer Science, National Taiwan University. <https://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf> (2003)

- [14] Singh, D., Singh, B.: Investigating the impact of data normalization on classification performance. *Applied Soft Computing* **97**, 105524 (2020) <https://doi.org/10.1016/j.asoc.2019.105524>
- [15] Smola, A.J., Schölkopf, B.: A tutorial on support vector regression. *Statistics and Computing* **14**(3), 199–222 (2004) <https://doi.org/10.1023/B:STCO.0000035301.49549.88>
- [16] Fernández-Delgado, M., Cernadas, E., Barro, S., Amorim, D.: Do we need hundreds of classifiers to solve real world classification problems? *Journal of Machine Learning Research* **15**, 3133–3181 (2014). <https://www.jmlr.org/papers/v15/delgado14a.html>
- [17] Chen, T., Guestrin, C.: XGBoost: A scalable tree boosting system. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, New York, NY, pp. 785–794 (2016). <https://doi.org/10.1145/2939672.2939785>
- [18] Meyer, H., Reudenbach, C., Hengl, T., Katurji, M., Nauss, T.: Improving performance of spatio-temporal machine learning models using forward feature selection and target-oriented validation. *Environmental Modelling & Software* **101**, 1–9 (2018) <https://doi.org/10.1016/j.envsoft.2017.12.001>
- [19] Dietterich, T.G.: Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation* **10**(7), 1895–1923 (1998) <https://doi.org/10.1162/089976698300017197>
- [20] Demšar, J.: Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research* **7**, 1–30 (2006). <https://www.jmlr.org/papers/v7/demsar06a.html>
- [21] Cawley, G.C., Talbot, N.L.C.: On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research* **11**, 2079–2107 (2010). <https://www.jmlr.org/papers/v11/cawley10a.html>
- [22] Shapley, L.S.: A value for n -person games. In: Kuhn, H.W., Tucker, A.W. (eds.) *Contributions to the Theory of Games II*, pp. 307–317. Princeton University Press, Princeton, NJ (1953). <https://doi.org/10.1515/9781400881970-018>
- [23] Drucker, H., Burges, C.J.C., Kaufman, L., Smola, A., Vapnik, V.: Support vector regression machines. In: *Advances in Neural Information Processing Systems* vol. 9, pp. 155–161. MIT Press, Cambridge, MA (1997). <https://papers.nips.cc/paper/1238-support-vector-regression-machines>
- [24] Vapnik, V.N.: *The Nature of Statistical Learning Theory*. Springer, New York (1995). <https://doi.org/10.1007/978-1-4757-2440-0>

- [25] Breiman, L.: Random forests. *Machine Learning* **45**(1), 5–32 (2001) <https://doi.org/10.1023/A:1010933404324>
- [26] Bergstra, J., Bengio, Y.: Random search for hyper-parameter optimization. *Journal of Machine Learning Research* **13**, 281–305 (2012). <https://www.jmlr.org/papers/v13/bergstra12a.html>
- [27] De Cock, D.: Ames, Iowa: Alternative to the Boston housing data as an end of semester regression project. *Journal of Statistics Education* **19**(3) (2011) <https://doi.org/10.1080/10691898.2011.11889627>
- [28] Chang, C.-C., Lin, C.-J.: LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology* **2**(3), 1–27 (2011) <https://doi.org/10.1145/1961189.1961199>
- [29] Williams, C.K.I., Seeger, M.: Using the Nyström method to speed up kernel machines. In: *Advances in Neural Information Processing Systems* vol. 13, pp. 682–688. MIT Press, Cambridge, MA (2001). <https://papers.nips.cc/paper/1866-using-the-nystrom-method-to-speed-up-kernel-machines>